Home Credit Default Risk: Modeling

Purpose

Home Credit wants to predict, at the time of application, which applicants are likely to have payment difficulties, so that it can reduce default losses and still approve creditworthy applicants with thin credit files. This notebook builds and compares models that estimate each applicant's probability of default, and it uses **AUC** (area under the ROC curve) as the yardstick. AUC measures how well a model ranks defaulters above non-defaulters: 0.5 is no better than a coin flip and 1.0 is a perfect ranking. It is the metric Kaggle uses for this competition, and unlike accuracy it is not fooled by the fact that only about 8% of applicants default.

The notebook follows the assignment steps: a benchmark, a comparison of candidate models, an experiment with class-imbalance strategies, randomized hyperparameter tuning, and a final model that produces a Kaggle submission. The modeling functions are defined in the collapsed code block directly below, and the cleaning and feature engineering functions come from `src/data_preparation.R` (the previous assignment).

```
library(data.table)
library(ggplot2)

# SMOKE = TRUE is a tiny, fast version I use only to test that the notebook runs
# from start to finish. Every reported result comes from the full version (SMOKE = F
SMOKE <- Sys.getenv("MODELING_SMOKE") == "1"

cfg <- list(
  compare_n = if (SMOKE) 6000 else 40000,   # rows for the model comparisons
  tune_n    = if (SMOKE) 3000 else 10000,   # rows for tuning
  n_search  = if (SMOKE) 3 else 20,         # random search iterations
  final_n   = if (SMOKE) 15000 else NA,     # NA = train the final model on all trai
  seed      = 2026
)

# Find the repo root. The notebook can run from notebooks/ or from the repo root
# (for example when I run chunks in Positron).
root <- if (file.exists("src/data_preparation.R")) "." else ".."
if (!file.exists(file.path(root, "src/data_preparation.R"))) {
  stop("Could not find src/data_preparation.R. Open the repo folder (the one that ",
       "contains src/, data/ and notebooks/) as your working directory. Current fold
}

# Slow steps are saved to disk (models/cache is git-ignored). Editing the text and
# re-rendering then skips the 40+ minutes of computation.
cache_dir <- file.path(root, if (SMOKE) "models/cache_smoke" else "models/cache")
dir.create(cache_dir, recursive = TRUE, showWarnings = FALSE)
```

```r
cached <- function(name, expr) {
  f <- file.path(cache_dir, paste0(name, ".rds"))
  if (file.exists(f)) return(readRDS(f))
  res <- expr                       # expr only runs when there is no saved result
  saveRDS(res, f)
  res
}

SKIP_PIPELINE <- TRUE   # load the functions only; skip the standalone pipeline
source(file.path(root, "src/data_preparation.R"))

train <- fread(file.path(root, "data/raw/application_train.csv"))
test  <- fread(file.path(root, "data/raw/application_test.csv"))
```

Modeling functions

The functions used throughout this notebook are defined here once so the sections below stay readable. Click “Show code” to expand them. The leakage rules they enforce are listed at the top of the block.

▶ Show the modeling functions

1. Evaluation design

The training file has 307,511 applicants and a default rate of 8.07%. The test file (48,744 applicants) has no labels, so the only way to estimate how a model will do on new applicants is **cross-validation** on the training data.

Three design choices keep those estimates honest:

1. **3-fold stratified cross-validation.** Each fold keeps the same default rate, and every number below is the average over the three folds, with the fold-to-fold standard deviation shown so that small differences can be judged against noise.
2. **Preparation is re-fit inside each fold.** The imputation medians and bin cut points from `fit_preprocessing()` are computed from that fold’s training rows only, then applied unchanged to its validation rows. This is the same fit-then-apply separation that `data_preparation.R` uses for train versus test, so the validation rows never influence how the training rows are prepared.
3. **Resampling touches training rows only.** When a class-imbalance strategy changes the training data (Section 4), the validation fold is left alone, so validation AUC always reflects the real 8% default rate.

To keep the many comparisons affordable on a laptop-class machine, the model comparison and class-imbalance experiments run on a stratified random sample of 40,000 training rows

(each fold trains on about 26,667 of them). Tuning uses a smaller 10,000-row sample, and only the final model sees all training rows.

```
comp_raw <- stratified_sample(train, cfg$compare_n, seed = cfg$seed)
cv_comp  <- make_cv_data(comp_raw, k = 3, seed = cfg$seed)

# Number of model inputs in each feature set
fold1 <- cv_comp[[1]]$train
n_feats <- sapply(c("ext_only", "raw", "engineered"), function(s) length(feature_set
n_feats
```

```
 ext_only       raw engineered
        3       120        135
```

2. Benchmarks

The **majority-class classifier** predicts “repaid” for everyone. It is the floor every real model has to beat.

```
bench_majority <- data.frame(
  model    = "Majority-class classifier (always predict 'repaid')",
  cv_auc   = fast_auc(train$TARGET, rep(0, nrow(train))),   # a constant score canno
  auc_sd   = NA_real_,
  accuracy = mean(train$TARGET == 0),
  seconds  = 0
)
knitr::kable(bench_majority[, c("model", "cv_auc", "accuracy")], digits = 4,
             col.names = c("Benchmark", "AUC", "Accuracy"))
```

Benchmark	AUC	Accuracy
Majority-class classifier (always predict ‘repaid’)	NA	0.9193

The majority classifier is 91.9% accurate while catching **zero** defaulters, which is why accuracy is the wrong yardstick for this problem and why every model below is judged on AUC. A better “simple model” benchmark is a logistic regression on only the three external credit scores (`EXT_SOURCE_1/2/3`), which is included as the first row of the comparison in Section 3.

3. Candidate models

The candidates differ in algorithm and in which predictors they see. The three predictor sets are:

- **External scores only** (3 columns): the simplest credible model.

- **Raw application columns** (120 columns): the original application fields, with the `DAYS_EMPLOYED = 365243` placeholder left in place.
- **Engineered features** (135 columns): the raw fields plus everything built in `data_preparation.R` (age in years, tenure with a placeholder flag, income and loan ratios, missing-data indicators, interactions, and bins). The raw `DAYS_BIRTH` and `DAYS_EMPLOYED` columns are dropped because the engineered versions replace them.

The algorithms are logistic regression (with an elastic-net penalty when there are many predictors, which shrinks unhelpful coefficients), random forests, and gradient boosting. Gradient boosting builds many small decision trees in sequence, each one correcting the errors of the previous ones. This session's R installation could not download `xgboost` or `lightgbm`, so gradient boosting uses the `gbm` package; it is the same family of model, only slower. Models start with reasonable default settings here and the best one is tuned in Section 5.

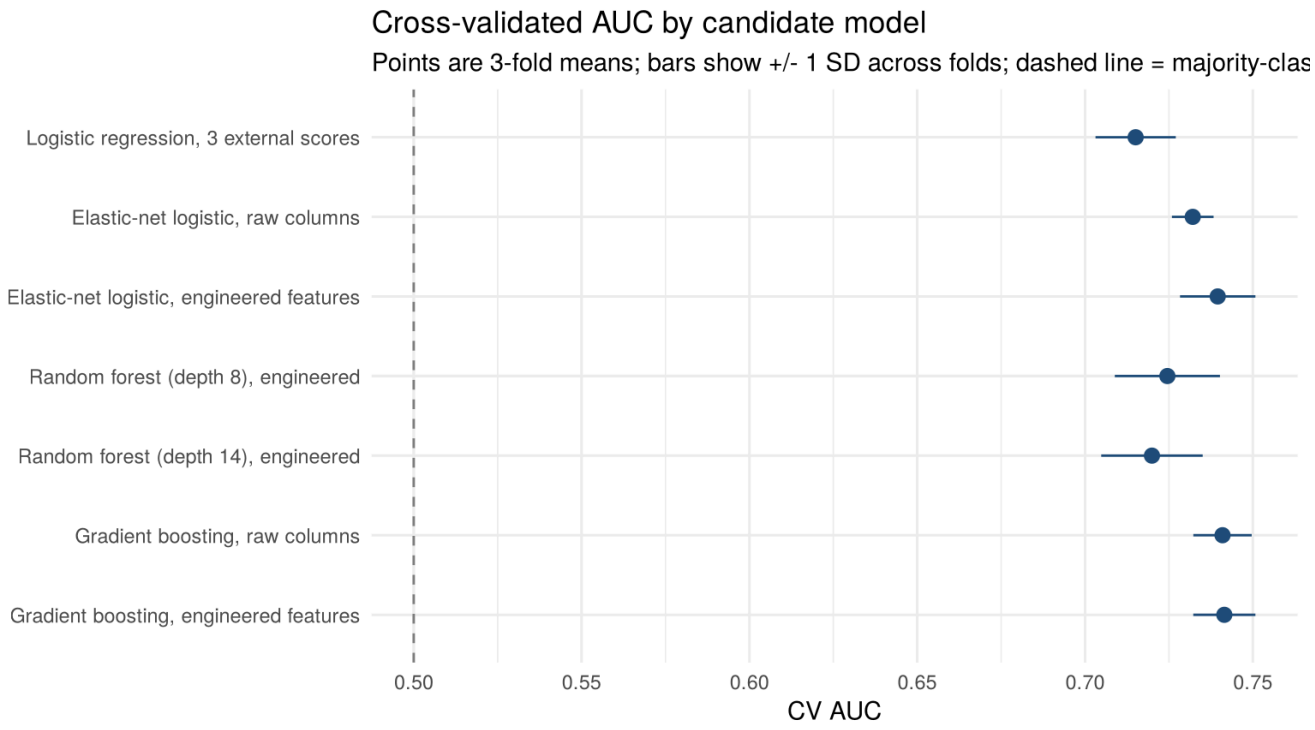
```
cand <- cached("candidates", {
  run <- function(label, fn, ...) {
    r <- cv_evaluate(cv_comp, fn, ...)
    list(summary = summarise_cv(r, label), folds = cbind(model = label, r))
  }
  runs <- list(
    run("Logistic regression, 3 external scores",       model_glm(),    features = "
    run("Elastic-net logistic, raw columns",            model_glmnet(), features = "
    run("Elastic-net logistic, engineered features",    model_glmnet(), features = "
    run("Random forest (depth 8), engineered",         model_ranger(num.trees = 300,
    run("Random forest (depth 14), engineered",        model_ranger(num.trees = 300,
    run("Gradient boosting, raw columns",               model_gbm(n.trees = 300), fe
    run("Gradient boosting, engineered features",       model_gbm(n.trees = 300), fe
  )
  list(summary = rbindlist(lapply(runs, `[[`, "summary")),
       folds   = rbindlist(lapply(runs, `[[`, "folds")))
})

comp_tbl <- rbind(bench_majority, cand$summary)
knitr::kable(comp_tbl[, c("model", "cv_auc", "auc_sd", "accuracy")], digits = 4,
             col.names = c("Model", "CV AUC", "SD across folds", "Accuracy at 0.5"))
```

Model	CV AUC	SD across folds	Accuracy at 0.5
Majority-class classifier (always predict 'repaid')	NA	NA	0.9193
Logistic regression, 3 external scores	0.7151	0.0120	0.9190
Elastic-net logistic, raw columns	0.7321	0.0062	0.9192
Elastic-net logistic, engineered features	0.7395	0.0112	0.9193

Model	CV AUC	SD across folds	Accuracy at 0.5
Random forest (depth 8), engineered	0.7245	0.0157	0.9193
Random forest (depth 14), engineered	0.7199	0.0151	0.9193
Gradient boosting, raw columns	0.7410	0.0087	0.9189
Gradient boosting, engineered features	0.7415	0.0093	0.9189

```
plot_df <- cand$summary
plot_df$model <- factor(plot_df$model, levels = rev(plot_df$model))
ggplot(plot_df, aes(x = cv_auc, y = model)) +
  geom_pointrange(aes(xmin = cv_auc - auc_sd, xmax = cv_auc + auc_sd), colour = "#1F
  geom_vline(xintercept = 0.5, linetype = "dashed", colour = "grey50") +
  labs(title = "Cross-validated AUC by candidate model",
       subtitle = "Points are 3-fold means; bars show +/- 1 SD across folds; dashed
       x = "CV AUC", y = NULL) +
  theme_minimal()
```




```
best_candidate <- cand$summary$model[which.max(cand$summary$cv_auc)]
best_candidate
```

```
[1] "Gradient boosting, engineered features"
```

Three results stand out. First, every model beats the benchmark's AUC of 0.5, and even a logistic regression on the three external scores reaches 0.715, so most of the signal is already

in those columns. Second, gradient boosting on the engineered features is best at 0.7415, but the elastic-net logistic regression is close at 0.7395 and gradient boosting on the raw columns is at 0.7410. The fold-to-fold standard deviation is about 0.009 to 0.011, which is larger than these gaps, so the top three models are not clearly separable on this sample. Third, the random forests are weaker (0.7245 and 0.7199), and the deeper forest is worse than the shallower one, which suggests that the forests overfit the noisy predictors. Accuracy is about 91.9% for every model and equals the majority-class benchmark, which confirms that accuracy cannot separate these models.

Gradient boosting is carried forward because it has the highest mean AUC and is the family that can use the most from tuning, not because it wins by a wide margin.

4. Class imbalance

Only about 8% of applicants default, so a model can do well on the overall loss by paying little attention to the rare class. The usual remedies change what the model sees during training:

- **Class weights:** keep every row but count each default as if it were about 11 defaults.
- **Downsampling:** keep every default and a random sample of repaid loans equal in number (1:1).
- **Upsampling:** randomly repeat defaults until they match the number of repaid loans (1:1).
- **SMOTE:** instead of repeating defaults, create synthetic ones by interpolating between neighbouring real defaults. SMOTE needs purely numeric inputs, so it is tested on the numeric columns only, against a numeric-only baseline.

All strategies are applied inside each training fold, never to the validation fold. They are tested on the two model families that performed best.

```
imb <- cached("imbalance", {
  strategies <- c("weights", "downsample", "upsample")
  one <- function(label, strategy, fn, ...) {
    cbind(strategy = strategy,
          summarise_cv(cv_evaluate(cv_comp, fn, imbalance = strategy, cores = 2, ...
  }
  gbm_rows   <- lapply(strategies, function(s) one("Gradient boosting", s, model_gbm
  glmnet_rows <- lapply(strategies, function(s) one("Elastic-net logistic", s, model
  smote_rows <- list(
    one("Elastic-net logistic, numeric columns only", "none",  model_glmnet(), featu
    one("Elastic-net logistic, numeric columns only", "smote", model_glmnet(), featu
  )
  rbindlist(c(gbm_rows, glmnet_rows, smote_rows))
})

# Add the unadjusted ("none") results from Section 3
none_rows <- cbind(strategy = "none", cand$summary[cand$summary$model %in%
```

```
  c("Gradient boosting, engineered features", "Elastic-net logistic, engineered feat
none_rows$model <- sub(", engineered features", "", none_rows$model)
imb_tbl <- rbind(none_rows, imb)
imb_tbl <- imb_tbl[order(imb_tbl$model, -imb_tbl$cv_auc), ]
knitr::kable(imb_tbl[, c("model", "strategy", "cv_auc", "auc_sd", "accuracy")], digi
             col.names = c("Model", "Imbalance strategy", "CV AUC", "SD across folds
```

Model	Imbalance strategy	CV AUC	SD across folds	Accuracy at 0.5
Elastic-net logistic	weights	0.7403	0.0106	0.6857
Elastic-net logistic	none	0.7395	0.0112	0.9193
Elastic-net logistic	upsample	0.7361	0.0074	0.6879
Elastic-net logistic	downsample	0.7341	0.0128	0.6805
Elastic-net logistic, numeric columns only	smote	0.7362	0.0100	0.6870
Elastic-net logistic, numeric columns only	none	0.7331	0.0106	0.9193
Gradient boosting	none	0.7415	0.0093	0.9189
Gradient boosting	upsample	0.7393	0.0073	0.7152
Gradient boosting	weights	0.7381	0.0081	0.7131
Gradient boosting	downsample	0.7326	0.0092	0.6694

Accuracy at a 0.5 cutoff drops sharply for the rebalanced models. That is expected and not a problem: after rebalancing, the model's probabilities are no longer calibrated to the real 8% default rate, so a 0.5 cutoff flags many more applicants. Only the *ranking* matters for AUC.

```
gbm_imb <- imb_tbl[imb_tbl$model == "Gradient boosting", ]
best_strategy <- gbm_imb$strategy[which.max(gbm_imb$cv_auc)]
best_strategy
```

```
[1] "none"
```

For gradient boosting, none of the imbalance strategies beat the unadjusted model (0.7415). Upsampling (0.7393) and class weights (0.7381) are slightly lower and downsampling (0.7326) is lowest, probably because it throws away most of the repaid loans. For the elastic-net logistic regression, weights are marginally higher than no adjustment (0.7403 vs 0.7395), and SMOTE is marginally higher than its numeric-only baseline (0.7362 vs 0.7331). Each of these differences is much smaller than the fold-to-fold standard deviation of 0.007 to 0.013, so they should be read as "no meaningful change". This is consistent with AUC being a ranking measure: reweighting the classes mostly changes how probabilities are scaled, not which applicants rank above which. Imbalance handling would matter more if the model were used with a fixed cutoff, and it would then be paired with recalibration. The unadjusted gradient-boosting model is used going forward.

5. Hyperparameter tuning

The best-performing family from Section 3 is tuned with a **randomized search**: instead of trying every combination on a grid, 20 random combinations are drawn from sensible ranges, and each is scored with 3-fold cross-validation. Following the assignment's speed-up guidance, the search runs on a random sample of only 10,000 training rows. For each sampled configuration the model is fit once with 400 trees and scored at 100, 200, 300 and 400 trees, so the number of trees is searched along with the other settings. The class-imbalance strategy chosen above (none) is held fixed.

Setting	What it controls	Range searched
`shrinkage`	how much each tree is allowed to change the model (learning rate)	0.03 to 0.20 (log scale)
`interaction.depth`	size of each tree	2 to 5
`n.minobsinnode`	smallest allowed group of applicants in a tree leaf	20, 50, 100, 200
`bag.fraction`	share of rows each tree sees	0.5 to 0.9
`n.trees`	number of trees	100, 200, 300, 400

```
tune_raw <- stratified_sample(train, cfg$tune_n, seed = cfg$seed + 1)
cv_tune  <- make_cv_data(tune_raw, k = 3, seed = cfg$seed + 1)

search <- cached("search",
  random_search_gbm(cv_tune, sample_gbm_params(cfg$n_search, seed = cfg$seed),
                    tree_grid = seq(100, 400, by = 100), imbalance = best_strategy,

knitr::kable(head(search[, c("iteration", "cv_auc", "n.trees", "shrinkage", "interac
                             "n.minobsinnode", "bag.fraction")], 5),
             digits = c(0, 4, 0, 3, 0, 0, 2), row.names = FALSE,
             caption = "Five best configurations from the random search (3-fold CV A
```

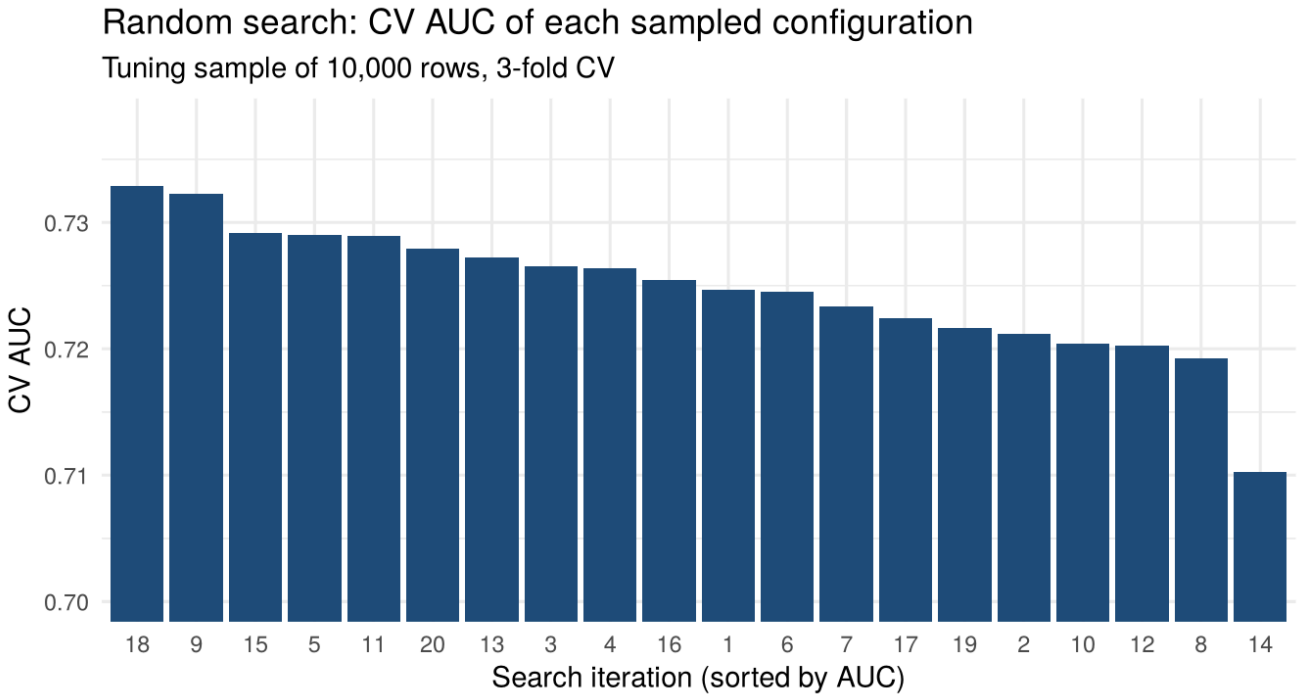
Five best configurations from the random search (3-fold CV AUC on the tuning sample)

iteration	cv_auc	n.trees	shrinkage	interaction.depth	n.minobsinnode	bag.fraction
18	0.7329	200	0.030	3	200	0.72
9	0.7323	200	0.048	2	200	0.73
15	0.7292	100	0.040	5	200	0.57

iteration	cv_auc	n.trees	shrinkage	interaction.depth	n.minobsinnode	bag.fraction
5	0.7290	100	0.086	3	200	0.66
11	0.7289	200	0.030	5	20	0.85

```
best <- search[1, ]
```

```
ggplot(search, aes(x = reorder(factor(iteration), -cv_auc), y = cv_auc)) +
  geom_col(fill = "#1F4E79") +
  coord_cartesian(ylim = c(min(search$cv_auc) - 0.01, max(search$cv_auc) + 0.005)) +
  labs(title = "Random search: CV AUC of each sampled configuration",
       subtitle = paste0("Tuning sample of ", format(cfg$tune_n, big.mark = ","), "
       x = "Search iteration (sorted by AUC)", y = "CV AUC") +
  theme_minimal()
```



A model tuned on a small sample can pick settings that do not carry over to more data, so the winning configuration is re-checked on the larger comparison folds from Section 3 (40,000 rows) against the default gradient-boosting settings under the same imbalance strategy.

```
tuned_fn <- model_gbm(n.trees = best$n.trees, interaction.depth = best$interaction.d
                      shrinkage = best$shrinkage, n.minobsinnode = best$n.minobsinno
                      bag.fraction = best$bag.fraction)

tuned_cv <- cached("tuned_check", cv_evaluate(cv_comp, tuned_fn, features = "enginee
                                              imbalance = best_strategy, cores = 2))

default_row <- gbm_imb[gbm_imb$strategy == best_strategy, ]
```

```
check_tbl <- rbind(
  data.frame(model = paste0("Gradient boosting, default settings (", best_strategy,
             cv_auc = default_row$cv_auc, auc_sd = default_row$auc_sd),
  data.frame(model = paste0("Gradient boosting, tuned settings (", best_strategy, ")
             cv_auc = mean(tuned_cv$auc), auc_sd = sd(tuned_cv$auc)))
knitr::kable(check_tbl, digits = 4, col.names = c("Model", "CV AUC", "SD across fold
```

Model	CV AUC	SD across folds
Gradient boosting, default settings (none)	0.7415	0.0093
Gradient boosting, tuned settings (none)	0.7381	0.0121

```
# Both models use the same three folds, so the fold differences are paired
default_folds <- cand$folds[cand$folds$model == "Gradient boosting, engineered featu
paired_diff   <- tuned_cv$auc - default_folds
```

Re-selecting the number of trees at a larger sample size

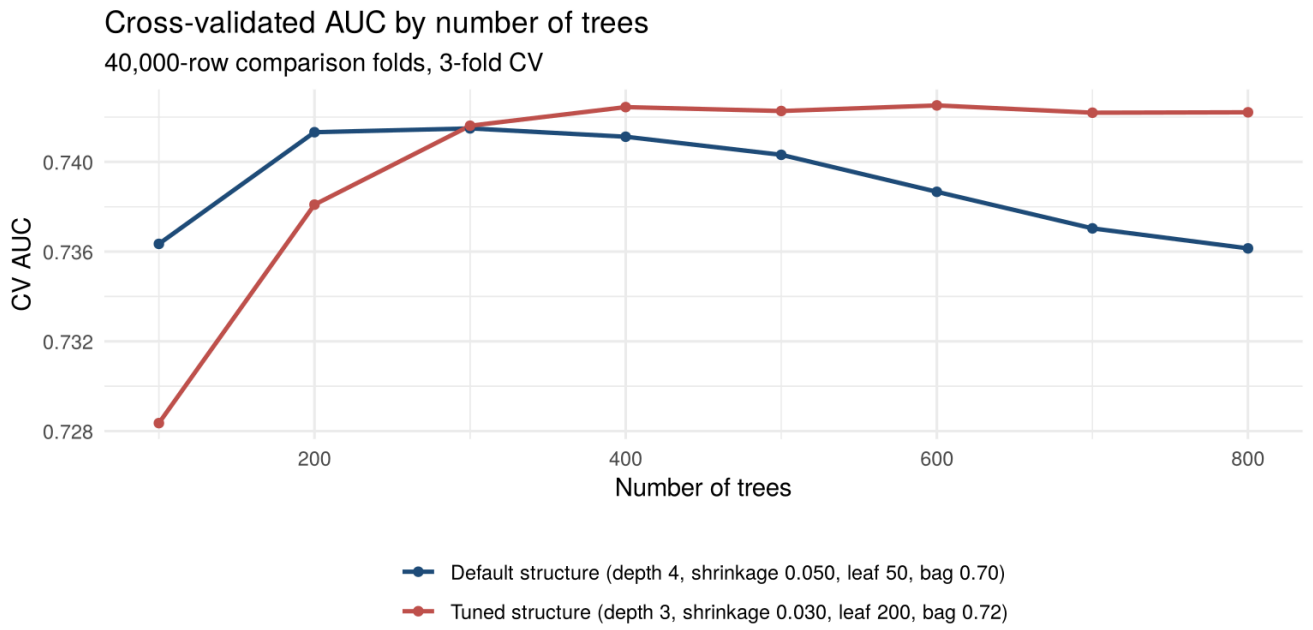
The check above is a warning sign: the settings that won on 10,000 rows do not beat the untuned defaults on the larger folds. The likely reason is the number of trees. With a low learning rate (`shrinkage`), a model needs enough trees to finish learning, and the right number grows with the amount of training data. A search on 10,000 rows can therefore favour a count that is too small for 27,000 or 307,000 rows. The fix is to keep the tuned *structure* (learning rate, tree size, leaf size, row sampling) and re-select only the number of trees on the larger comparison folds. The same curve is drawn for the default structure so the two can be compared fairly.

```
default_params <- data.frame(shrinkage = 0.05, interaction.depth = 4, n.minobsinnode
tuned_label   <- sprintf("Tuned structure (depth %d, shrinkage %.3f, leaf %d, bag %.
                         best$interaction.depth, best$shrinkage, best$n.minobsinnode
default_label <- "Default structure (depth 4, shrinkage 0.050, leaf 50, bag 0.70)"

curves <- cached("tree_curves", {
  grid <- seq(100, 800, by = 100)
  rbind(
    cbind(config = default_label, gbm_tree_curve(cv_comp, default_params, grid, imba
    cbind(config = tuned_label,   gbm_tree_curve(cv_comp, best,           grid, imba
  )
})
```

```
ggplot(curves, aes(x = n.trees, y = cv_auc, colour = config)) +
  geom_line(linewidth = 0.9) + geom_point() +
  scale_colour_manual(values = c("#1F4E79", "#C0504D")) +
  labs(title = "Cross-validated AUC by number of trees",
```

```
       subtitle = paste0(format(cfg$compare_n, big.mark = ","), "-row comparison fol
       x = "Number of trees", y = "CV AUC", colour = NULL) +
  theme_minimal() + theme(legend.position = "bottom", legend.direction = "vertical")
```




```
top <- curves[which.max(curves$cv_auc), ]
final_params <- if (top$config == default_label) default_params else best
final_trees  <- top$n.trees
final_fn <- model_gbm(n.trees = final_trees, interaction.depth = final_params$intera
                      shrinkage = final_params$shrinkage, n.minobsinnode = final_par
                      bag.fraction = final_params$bag.fraction)

knitr::kable(data.frame(
  Setting = c("Structure", "Number of trees", "CV AUC (comparison folds)", "SD acros
  Value   = c(top$config, final_trees, sprintf("%.4f", top$cv_auc), sprintf("%.4f",
```

Setting	Value
Structure	Tuned structure (depth 3, shrinkage 0.030, leaf 200, bag 0.72)
Number of trees	600
CV AUC (comparison folds)	0.7425
SD across folds	0.0102

At 200 trees the tuned structure scored 0.7381, below the default model's 0.7415. With the tree count re-selected, the tuned structure (depth 3, shrinkage 0.030, leaf size 200, bag fraction 0.72) peaks at 600 trees with a CV AUC of 0.7425, compared to 0.7415 for the default structure at 300 trees. The tuned curve is also flatter: it stays within 0.0003 of its peak from 400 to 800 trees, while the default structure falls off after 300 trees as it begins to overfit. So the benefit

of tuning is a gain of about 0.001 AUC and a model that is less sensitive to the tree count. That gain is far inside the noise, and the honest reading is that the default settings were already close to what this data allows. The tuned configuration at 600 trees is used as the final model because it has the highest mean AUC and the most stable curve.

6. Supplementary tables

The assignment makes this step optional. The data-preparation script from the previous assignment aggregates only the two application tables, and the supplementary tables (`bureau.csv`, `previous_application.csv`, `installments_payments.csv`, and the others) were not available in this environment, so **no supplementary features are used and no with/without comparison is reported**. Everything in this notebook is therefore a model built from application data alone. The natural next step would be to aggregate each supplementary table to one row per applicant, left-join it onto the application table, and rerun Sections 3 and 5 with and without the new columns.

7. Final model and Kaggle submission

The final model uses the gradient-boosting configuration selected at the end of Section 5 and the class-imbalance strategy chosen in Section 4. It is trained on **all** training rows. Preparation statistics are fit on the training data once and then applied, unchanged, to the test data; `fit_final_and_predict()` stops with an error if the prepared train and test designs differ in any column or category level.

```
final_train <- if (is.na(cfg$final_n)) train else stratified_sample(train, cfg$final

final <- cached("final", fit_final_and_predict(final_train, test, final_fn, features
                                               imbalance = best_strategy, seed = cf

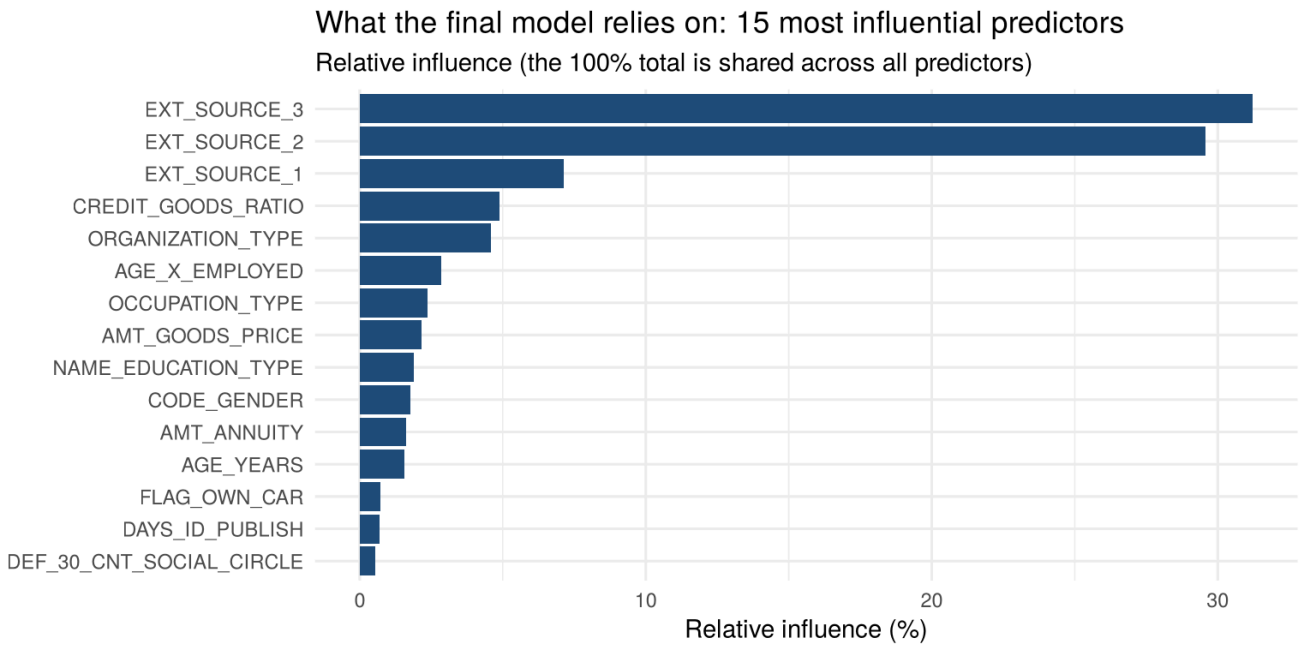
sub <- data.table(SK_ID_CURR = test$SK_ID_CURR, TARGET = final$test_pred)
stopifnot(nrow(sub) == nrow(test), !anyNA(sub$TARGET), all(sub$TARGET >= 0 & sub$TAR
          identical(sub$SK_ID_CURR, test$SK_ID_CURR))

dir.create(file.path(root, "submissions"), showWarnings = FALSE)
sub_file <- file.path(root, "submissions", if (SMOKE) "submission_smoke.csv" else "s
fwrite(sub, sub_file)
```

The submission file has 48,744 rows (one per test applicant) and two columns, `SK_ID_CURR` and `TARGET` (the predicted probability of default), which is the format Kaggle requires. The model was trained on 307,511 rows and 135 predictors.

```
imp <- head(final$importance[order(-final$importance$influence), ], 15)
imp$feature <- factor(imp$feature, levels = rev(imp$feature))
```

```
ggplot(imp, aes(x = influence, y = feature)) +
  geom_col(fill = "#1F4E79") +
  labs(title = "What the final model relies on: 15 most influential predictors",
       subtitle = "Relative influence (the 100% total is shared across all predictor
       x = "Relative influence (%)", y = NULL) +
  theme_minimal()
```



Kaggle score

```
# Score from uploading submissions/submission.csv to Kaggle:
KAGGLE_SCORE <- 0.74224       # public leaderboard score (AUC)
cv_estimate  <- top$cv_auc
```

- **Cross-validated AUC estimate for this configuration:** 0.7425 (3-fold, on 40,000 rows). The final model is trained on all 307,511 rows, so the leaderboard score can differ from this estimate.
- **Kaggle public score:** 0.7422

8. Sensitivity check: demographic fields

The feature report flagged four fields as needing a fair-lending review before use in a credit decision: `CODE_GENDER`, `NAME_FAMILY_STATUS`, `CNT_CHILDREN` and `CNT_FAM_MEMBERS`. The models above include them, because they are ordinary application columns and Kaggle scores only predictive accuracy. Since the business problem statement lists fairness review as a

constraint, it is worth knowing how much predictive accuracy depends on them. The final configuration is re-run on the same folds without those four columns.

```
demog_cv <- cached("demog_check", cv_evaluate(cv_comp, final_fn, features = "enginee
                                              imbalance = best_strategy, cores = 2))
demog_tbl <- data.frame(
  model  = c("Final configuration, all engineered features", "Final configuration, d
  cv_auc = c(top$cv_auc, mean(demog_cv$auc)),
  auc_sd = c(top$auc_sd, sd(demog_cv$auc)))
knitr::kable(demog_tbl, digits = 4, col.names = c("Model", "CV AUC", "SD across fold
```

Model	CV AUC	SD across folds
Final configuration, all engineered features	0.7425	0.0102
Final configuration, demographic fields removed	0.7412	0.0096

Removing the four fields lowers CV AUC from 0.7425 to 0.7412, a difference of 0.0013 that is well inside the fold-to-fold noise. This suggests that the model does not depend on these fields for its predictive power, which matters for a lender that has to defend the model on fair-lending grounds. The submitted model keeps them, because the assignment is scored on accuracy only, but a production version could drop them at almost no cost in AUC.

9. Summary

Models tried. A majority-class benchmark (AUC 0.5, accuracy 91.93%), logistic regression on the three external scores, elastic-net logistic regression on raw and engineered features, two random forests, and gradient boosting on raw and engineered features. All were scored with 3-fold cross-validated AUC.

Result. The best model is gradient boosting on the engineered features with a tuned structure and 600 trees. Its cross-validated AUC is 0.7425 on 40,000 rows, compared to 0.5 for the benchmark and 0.7151 for logistic regression on the external scores alone.

Why this model. It has the highest mean AUC, and it is the most stable as the number of trees grows. The margin is small. The top three candidates, the three imbalance-handling results for gradient boosting, and the tuned and default settings are all within about one standard deviation of each other, so the choice among them is closer to a judgment call than to a clear win. Gradient boosting was preferred to the logistic regression because it can capture interactions without hand-built terms, and because its lead, though small, held across the raw features, engineered features and the tuning checks.

What did not help. Class-imbalance adjustments did not raise AUC for gradient boosting. The random forests were weaker than boosting. Adding engineered features helped only slightly

(0.7415 compared to 0.7410 for raw columns in boosting, and 0.7395 compared to 0.7321 in the elastic net).

What drives the predictions. The three external scores account for 68% of the final model's relative influence. This suggests that the model is mostly a rule for combining those scores, with smaller contributions from credit-to-goods ratio and organization type.

Limits. The comparison used 40,000 of the 307,511 training rows, and the final model was trained on all of them. The Kaggle public score of 0.7422 is close to the cross-validated estimate of 0.7425, so training on more rows did not raise the score by much and the cross-validation did not look optimistic. Supplementary tables were not used, and these usually carry substantial signal (bureau and repayment history), so adding them is the most likely route to a higher score.